



**Addis Ababa Science and Technology University**  
School of Electrical Engineering and Computing  
Department of Software Engineering

---

# **BankruptcySense AI**

## **Technical Documentation**

Bankruptcy Prediction System for Small Businesses

---

---

| Model    | Random Forest + SMOTE + OOF Threshold Tuning |
|----------|----------------------------------------------|
| Dataset  | Polish Bankruptcy Dataset (UCI) — 5year.arff |
| Frontend | bankruptcy-sense-ai.vercel.app               |
| Backend  | bankruptcysense-ai.onrender.com              |

GROUP MEMBERS

| No. | Full Name      | Student ID  |
|-----|----------------|-------------|
| 1.  | SISIYO TAKELE  | ETS 1252/15 |
| 2.  | SOLAN ADUGNA   | ETS1260/15  |
| 3.  | YODIT GIRMA    | ETS 1422/15 |
| 4.  | YEABSERA YIDEG | ETS 1399/15 |
| 5.  | TSEGAYE BERHE  | ETS 1437/15 |
| 6.  | RUTH AREDA     | ETS 1519/15 |

Academic Year: 2024/2025  
Course: Machine Learning Project  
*github.com/sisiyotakele/BankruptcySense-AI*

---

## 1. Project Overview

BankruptcySense AI is a machine-learning system that predicts bankruptcy risk for small businesses using financial ratio data. Given a set of ratios covering profitability, liquidity, leverage, and operational efficiency, the model outputs a probability score and a risk tier — together giving analysts a fast, data-driven signal on whether a company deserves closer scrutiny.

The core challenge here is not the modelling itself but the data reality: bankrupt companies make up only about 7% of the Polish dataset. That kind of imbalance kills naive classifiers — a model that always says "safe" would be right 93% of the time while being completely useless. Everything in this pipeline, from SMOTE resampling to OOF threshold tuning, is built around that constraint.

### Problem Framing

| Attribute         | Description                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------|
| Type              | Binary classification — 0 (Safe) vs 1 (Bankrupt)                                               |
| Primary Metric    | Recall on the bankrupt class — missing a real bankruptcy is far more costly than a false alarm |
| Secondary Metrics | ROC-AUC (discriminative power), F1 (overall balance), CV Std (stability)                       |

### Technology Stack

| Layer       | Technology                      | Version                |
|-------------|---------------------------------|------------------------|
| ML Core     | Scikit-learn + imbalanced-learn | 1.6.1 / 0.14.1         |
| Data        | Pandas, NumPy, SciPy            | 2.2.3 / 2.2.6 / 1.15.3 |
| Runtime     | Python                          | 3.13                   |
| Backend API | Flask + Gunicorn                | 3.0.1 / 21.2.0         |
| Frontend    | React 18 + Vite 5 + Tailwind 3  | —                      |
| Hosting     | Render (API) + Vercel (UI)      | —                      |

## 2. Dataset Analysis

### Source

The dataset is the 5year.arff file from the Polish Companies Bankruptcy Dataset (UCI Machine Learning Repository). It contains financial data for 5,910 Polish companies, each described by 64 financial ratios measured in the fifth year before the potential bankruptcy event.

### Dataset at a Glance

| Property                          | Value                               |
|-----------------------------------|-------------------------------------|
| Total companies                   | 5,910                               |
| Financial features (Attr1–Attr64) | 64 continuous ratios                |
| Bankrupt companies                | ~410 (6.94%)                        |
| Safe companies                    | ~5,500 (93.06%)                     |
| Missing value columns             | Present in many; Attr37 exceeds 40% |

### The Class Imbalance Problem

A 13:1 safe-to-bankrupt ratio is the defining challenge of this dataset. Accuracy becomes a misleading metric almost immediately — a classifier that never predicts bankruptcy scores 93% accuracy while catching zero actual failures. The only metrics that matter here are recall on the bankrupt class, F1, and ROC-AUC.

Key insight: precision and recall are in fundamental tension on this dataset. Catching more bankruptcies (higher recall) inevitably means flagging more safe companies as risky (lower precision). Every design choice in this pipeline reflects a deliberate position on that trade-off.

### Selected Feature Descriptions

| Feature | Description                      |
|---------|----------------------------------|
| Attr1   | Net profit / Total assets        |
| Attr6   | Retained earnings / Total assets |
| Attr13  | EBIT / Total assets              |
| Attr15  | Total liabilities / Total assets |

|        |                                                       |
|--------|-------------------------------------------------------|
| Attr21 | Net profit / Sales (net margin)                       |
| Attr35 | Total liabilities / (Operating profit + Depreciation) |
| Attr39 | Profit on operating activities / Total assets         |
| Attr41 | Total liabilities / (Net profit + Depreciation)       |

## Missing Data Strategy

Attr37 was removed entirely — more than 40% of its values are absent, and imputing from the remaining 60% would introduce more noise than signal. Every other column was imputed with the column median computed from the training split only. Median imputation was chosen over mean because financial ratios are skewed distributions; a single distressed company can produce extreme outlier values that would pull the mean far from the typical company profile.

## 3. ML Pipeline — Step-by-Step

### 3.1 Why the Order of Operations Matters

Data leakage is the single most common source of over-optimistic results in applied ML. It happens when information from the test set — even indirectly — influences how the model is trained. In this pipeline, the test set is sealed immediately after the stratified split and never touched again until final evaluation.

The correct order is:

```

Raw data
├─ Drop Attr37          (structural decision — OK before split)
├─ STRATIFIED SPLIT ← test set sealed here
│   ├── Impute median    (fit on train → apply to both)
│   ├── Clip outliers    (1st/99th pctile from train → clip both)
│   ├── StandardScaler   (fit on train → transform both)
│   ├── SMOTE            (train only — test stays real)
│   ├── Feature selection (fit on SMOTE train → select both)
│   ├── Hyperparameter tuning (train only)
│   ├── OOF threshold tuning (OOF on original train)
│   └─ Evaluate          (test set — first and only time)

```

## What Goes Wrong Without This Order

| Wrong Step                     | Consequence                                                       |
|--------------------------------|-------------------------------------------------------------------|
| Scale before split             | Test means contaminate the scaler → inflated results              |
| SMOTE before split             | Synthetic samples derived from test distribution → severe leakage |
| Feature selection before split | Importance scores include test signal → selection bias            |
| Threshold tuning on test       | Direct optimisation on test data → no honest evaluation possible  |

### 3.2 Stratified Train/Test Split

An 80/20 split produces 4,728 training rows and 1,182 test rows. The `stratify=y` parameter ensures both halves maintain the 6.94% bankrupt rate. Without stratification, random splits can produce test sets with noticeably different class ratios, making evaluation unreliable — particularly bad when the minority class is already small.

### 3.3 Median Imputation

All remaining missing values are filled with the per-column median, computed on the training set and applied identically to the test set. The median is preferred over mean for financial ratios because extreme outlier companies would skew the mean substantially, giving a distorted picture of what a "typical" value looks like for each ratio.

### 3.4 Outlier Clipping

Values below the 1st percentile or above the 99th percentile (both derived from training data) are clipped to those bounds. The clip bounds are saved to `clip_bounds.pkl` and applied at inference time. Outliers are not removed — they represent real companies with genuinely extreme ratios — but their influence on the scaler and tree splits is contained.

A subtle correctness issue in an earlier version of this code: the clipping bounds were recomputed from  $\pm 3\sigma$  of the already-fitted scaler at inference time. Because the scaler is fit on already-clipped data, those bounds differed from the original training bounds, creating silent training-serving skew. The fix is to save the original percentile bounds explicitly and reload them.

### 3.5 StandardScaler

Each feature is normalised to zero mean and unit variance. Random forests are technically scale-invariant — they split on thresholds rather than distances — so this step is not strictly required for tree performance. It is included for three practical reasons: it makes the SMOTE synthetic samples behave more predictably in feature space, it stabilises the OOF probability outputs across folds, and it future-proofs the pipeline if a distance-based model is later introduced.

The scaler is fit on all 63 remaining features (after dropping Attr37). At inference time, inputs pass through this scaler before feature selection reduces to 30. This means scaler.pkl always expects exactly 63 features.

### 3.6 SMOTE — Handling Class Imbalance

With only 328 bankrupt companies in the training split (~6.94%), a Random Forest trained without any correction will be strongly biased toward the "Safe" prediction. It minimises overall loss by essentially ignoring the minority class.

SMOTE (Synthetic Minority Over-sampling Technique) addresses this by generating new synthetic bankrupt samples through interpolation:

```
For each minority sample  $x_i$ :
  1. Find its  $k$  nearest neighbours among other bankrupt samples
  2. Randomly select one neighbour  $x_j$ 
  3. Create:  $x_{\text{new}} = x_i + \lambda(x_j - x_i)$ 
     where  $\lambda \in [0, 1]$ 
```

This is meaningfully different from random oversampling (which just copies existing points and causes overfitting). SMOTE creates plausible new examples that lie on the manifold of the minority class.

#### Before and After SMOTE

|              | Safe (Class 0) | Bankrupt (Class 1) |
|--------------|----------------|--------------------|
| Before SMOTE | 4,400          | 328                |
| After SMOTE  | 4,400          | 4,400              |

SMOTE is applied to training data only. Applying it before the train/test split would generate synthetic samples derived from the test distribution — the model would then be evaluated on data it has effectively already seen, inflating recall dramatically.

### Why Not Other Imbalance Approaches?

| Approach             | Reason Not Used                                                                   |
|----------------------|-----------------------------------------------------------------------------------|
| Class weights alone  | Insufficient for 13:1 imbalance on its own; SMOTE adds more signal                |
| Random oversampling  | Simply duplicates existing points — causes overfitting                            |
| Undersampling        | Discards real majority samples; wastes already-limited data                       |
| ADASYN               | Adds complexity without a clear benefit at this dataset scale                     |
| Threshold shift only | Adjusts the decision boundary but does not help the model learn minority patterns |

SMOTE and `class_weight="balanced"` are used together. SMOTE operates at the data level; `class_weight` operates at the loss level. Using both provides stronger protection against majority-class bias than either does alone.

### 3.7 Feature Selection

Starting from 63 features (after dropping Attr37), a preliminary Random Forest is trained on the SMOTE-resampled data to rank all features by their Gini impurity decrease. The top 30 are kept. This reduces noise from correlated or redundant ratios, speeds up inference, and produces more calibrated probability outputs.

#### Top 5 Most Important Features

| Rank | Feature | Description                                     | Importance |
|------|---------|-------------------------------------------------|------------|
| 1    | Attr39  | Profit on operating activities / Total assets   | ~10.0%     |
| 2    | Attr35  | Total liabilities / (Op. profit + Depreciation) | ~10.0%     |
| 3    | Attr21  | Net profit / Sales (net margin)                 | ~7.2%      |
| 4    | Attr41  | Total liabilities / (Net profit + Depreciation) | ~6.0%      |
| 5    | Attr27  | Operating profit / Financial expenses           | ~6.0%      |



These features tell a coherent story about bankruptcy risk: how efficiently a company converts assets into profit (Attr39), whether it can service its debt load (Attr35, Attr41), and whether its core operations are profitable at all (Attr21, Attr27). The top 30 features collectively explain about 85% of the total feature importance mass.

### 3.8 Hyperparameter Tuning

Tuning uses RandomizedSearchCV over 30 random combinations with 5-fold StratifiedKFold cross-validation, optimising directly for recall on the minority class. Grid search was considered but rejected: the search space has hundreds of combinations, making random sampling  $\sim 20\times$  faster for comparable coverage.

#### Search Space

| Parameter                | Values Tested                | Rationale                                                         |
|--------------------------|------------------------------|-------------------------------------------------------------------|
| <b>n_estimators</b>      | 100, 200, 300                | More trees = more stable; diminishing returns past $\sim 200$     |
| <b>max_depth</b>         | 10, 20, 30, None             | Unlimited depth is fine here given SMOTE regularisation           |
| <b>min_samples_split</b> | 2, 5, 10                     | Higher values reduce overfitting at split points                  |
| <b>min_samples_leaf</b>  | 1, 2, 4                      | Higher values smooth decision boundaries                          |
| <b>max_features</b>      | sqrt, log2, 0.3              | Controls tree diversity; 0.3 adds a mid-range option              |
| <b>class_weight</b>      | balanced, balanced_subsample | Both correct for imbalance; subsample variant recomputes per tree |

#### Best Parameters Found

```
n_estimators=200
max_depth=None (unlimited)
min_samples_split=2
min_samples_leaf=1
max_features=0.3
class_weight="balanced"
Best CV recall (on SMOTE-balanced data): 0.9789
```

### 3.9 OOF Threshold Tuning

The default decision threshold of 0.5 is designed for balanced datasets. At a 7% base rate, a predicted probability of 0.3 can represent a genuinely meaningful signal that a company is at risk — but the default threshold would classify it as safe. Threshold tuning corrects this.

## Why Not Tune on the Test Set?

A threshold tuned on the test set would be optimised for that specific sample and would not generalise. The test set must remain untouched until the final evaluation.

## The OOF Approach

The solution is to generate out-of-fold probability estimates on the original (pre-SMOTE) training data. In each of 5 folds, SMOTE is applied only to the training portion of that fold, a proxy model is trained, and probabilities are predicted on the held-out fold (real, imbalanced data). The resulting OOF probabilities reflect the real class distribution rather than the synthetic 50/50 one.

```
for each fold (5-fold StratifiedKfold):  
    └─ Apply SMOTE to fold training portion  
    └─ Train 100-tree proxy model on resampled fold  
    └─ Predict probabilities on held-out fold (real distribution)  
Combine all held-out predictions → OOF probability array  
Search precision-recall curve for optimal threshold
```

## Threshold Selection Logic

The threshold search enforces two constraints: recall  $\geq 0.75$  (catching at least 75% of real bankruptcies) and precision  $\geq 0.20$  (avoiding completely indiscriminate flagging). Within that feasible region, the threshold that maximises F1 is selected. If no single threshold satisfies both constraints, precision is relaxed while recall is preserved — missing bankruptcies is the costlier error.

| Result                     | Value |
|----------------------------|-------|
| Optimal threshold          | 0.29  |
| OOF recall at threshold    | 0.76  |
| OOF precision at threshold | 0.30  |
| OOF F1 at threshold        | 0.43  |

A threshold of 0.29 means any company with a predicted bankruptcy probability of 29% or higher is classified as Bankrupt. This is substantially below 0.5 because the 7% base rate means even moderate probability signals are informative relative to the prior.

## 4. Model Evaluation & Results

All metrics below are computed on the held-out test set of 1,182 companies. This set was sealed before training, SMOTE, feature selection, and threshold tuning — it represents the first honest look at model performance on real unseen data.

### 4.1 Final Test-Set Metrics

| Metric                    | Result | Target / Status       |
|---------------------------|--------|-----------------------|
| ROC-AUC                   | 0.9121 | > 0.90 ✓ PASS         |
| Recall (bankrupt class)   | 0.7927 | > 0.75 ✓ PASS         |
| F1 Score (bankrupt class) | 0.4610 | > 0.72 ✗ BELOW TARGET |
| CV Std (across 5 folds)   | 0.0045 | < 0.05 ✓ PASS         |

### 4.2 Confusion Matrix

|                  | Predicted: Safe     | Predicted: Bankrupt  |
|------------------|---------------------|----------------------|
| Actual: Safe     | 965 (True Negative) | 135 (False Positive) |
| Actual: Bankrupt | 17 (False Negative) | 65 (True Positive)   |

The model correctly identified 65 of the 82 bankrupt companies in the test set (recall 79.3%), while raising 135 false alarms among 1,100 safe companies. The 17 missed bankruptcies (false negative rate 20.7%) represent the most costly errors — these are the companies that would slip through without being flagged.

### 4.3 Why F1 < 0.72 Is Not a Modelling Failure

F1 of 0.72 is ambitious for this dataset. Published research on the Polish bankruptcy dataset consistently reports F1 in the 0.40–0.65 range for the bankrupt class. The fundamental reason is arithmetic: at 7% base rate with recall of ~0.79, achieving F1 of 0.72 requires precision above 0.65. But catching 79% of bankruptcies while keeping precision that high is not possible without a much larger and cleaner dataset.

The ROC-AUC of 0.912 tells the more important story. It means the model correctly ranks a randomly chosen bankrupt company above a randomly chosen safe company 91.2% of the

time — regardless of where the threshold is set. That is a strong discriminative ability, and it is the right metric to judge a model that will be used with different risk tolerances in practice.

#### 4.4 Precision–Recall Trade-off Across Thresholds

| Threshold       | Recall | Precision / F1 |
|-----------------|--------|----------------|
| 0.16            | 0.90   | 0.20 / 0.33    |
| 0.25            | 0.85   | 0.27 / 0.40    |
| 0.29 (selected) | 0.79   | 0.33 / 0.46    |
| 0.40            | 0.67   | 0.42 / 0.52    |
| 0.50            | 0.49   | 0.53 / 0.51    |

F1 peaks around 0.52 at threshold 0.40–0.50, but at a substantial recall cost. The selected threshold of 0.29 accepts lower F1 to maintain the recall floor — consistent with the design goal of catching as many bankruptcies as possible.

## 5. Key Design Decisions & Trade-offs

### Scoring Metric: Recall Over F1

RandomizedSearchCV is configured to optimise recall directly. The business rationale is asymmetric cost: an investor who misses a bankrupt company loses their full investment; one who flags a safe company loses time on due diligence. Optimising for recall pushes the model to favour sensitivity over precision, which is appropriate here. The consequence is a tuned model that catches 79% of bankruptcies while generating about 135 false alarms per 1,182 companies — a trade-off accepted by design.

### SMOTE + class\_weight: Two Layers of Imbalance Correction

Both are used simultaneously. SMOTE adds synthetic minority-class samples at the data level, making the training distribution more balanced. `class_weight="balanced"` adds a penalty at the loss level, so any residual imbalance — caused by bootstrapping in individual trees — is further corrected. Either approach alone is insufficient for a 13:1 ratio; together they provide robust coverage.

## OOF Threshold vs Training-Set Threshold

Searching for the best threshold on the full training set fails because the model has already memorised that data — its probability outputs are overconfident and the optimal threshold found there does not transfer to unseen data. OOF probabilities come from fold-trained models that have never seen the validation samples, producing honest probability estimates that calibrate correctly to the real class distribution. This is the reason the threshold of 0.29 found through OOF actually works at test time.

## TOP\_N\_FEATURES = 30 (Not 20)

The initial plan was 20 features. After reviewing importance scores, features ranked 21–30 each contributed roughly 2–4% of cumulative importance. Keeping 30 retains about 85% of all feature importance mass, versus ~75% for 20. The marginal inference cost of 10 extra features is negligible, and the additional signal measurably improved recall on held-out validation.

## Why Not Deep Learning?

At ~5,900 rows and 63 features, this is a small tabular dataset. Neural networks require substantially more data to learn stable representations, and their training cost is not justified when tree-based methods consistently outperform them at this scale — a finding replicated across multiple published studies on tabular data (Grinsztajn et al., 2022). Random Forest was the correct tool: interpretable, fast to train and tune, naturally robust to outliers, and capable of capturing non-linear interactions between financial ratios.

## Inference Pipeline Consistency (Training-Serving Skew)

The most operationally important constraint in the entire pipeline is that the inference code must mirror the training code exactly. Any discrepancy — different clip bounds, different feature ordering, a missing imputation step — produces inputs the model has never seen and degrades performance silently.

| Training Step                         | Inference Equivalent                       |
|---------------------------------------|--------------------------------------------|
| Drop Attr37                           | Never include Attr37 in input              |
| Fill NaN with column median           | Fill missing with 0 (neutral post-scaling) |
| Clip to [1st, 99th] pctile from train | Clip using saved clip_bounds.pkl           |
| Scale with fitted StandardScaler      | Apply saved scaler.pkl                     |

|                               |                                      |
|-------------------------------|--------------------------------------|
| Select top-30 feature indices | Apply same indices from features.pkl |
|-------------------------------|--------------------------------------|

## 6. Saved Model Artifacts

Training produces five files. All five must be present and version-consistent for inference to work correctly. They are committed to the repository and deployed to Render at startup — no training happens in production.

| File                   | Size    | Contents                                                | Used By    |
|------------------------|---------|---------------------------------------------------------|------------|
| <b>rf_model.pkl</b>    | ~13 MB  | Trained RandomForestClassifier (200 trees)              | predict.py |
| <b>scaler.pkl</b>      | ~3 KB   | StandardScaler — $\mu$ and $\sigma$ for all 63 features | predict.py |
| <b>features.pkl</b>    | ~0.3 KB | List of 30 selected feature names by index              | predict.py |
| <b>clip_bounds.pkl</b> | ~1 KB   | Dict {lower, upper} — 1st/99th pctile per feature       | predict.py |
| <b>threshold.pkl</b>   | ~0.1 KB | Float 0.29 — OOF-tuned decision threshold               | predict.py |

All five artifacts are loaded once at Flask startup into module-level variables. The ~13 MB model is therefore in memory before the first request arrives, keeping per-request inference latency at 5–20 ms.

## 7. Backend API Reference

Base URL: <https://bankruptcysense-ai.onrender.com>

### Endpoints

| Endpoint       | Method | Description                                         |
|----------------|--------|-----------------------------------------------------|
| /              | GET    | Root — returns API info and available endpoints     |
| /health        | GET    | Liveness check → {"status":"ok","model":"loaded"}   |
| /features      | GET    | Returns the 63 valid feature names                  |
| /predict       | POST   | Single prediction — full result dict with risk tier |
| /predict/batch | POST   | Batch prediction for up to 500 rows                 |

|          |        |                                            |
|----------|--------|--------------------------------------------|
| /history | GET    | Recent predictions (newest first, max 200) |
| /history | DELETE | Clear prediction history                   |

## Prediction Request

Only include the features you have. Any missing feature is treated as unknown and imputed with 0 (neutral after scaling). Unknown keys are rejected by the validator.

```
POST /predict
Content-Type: application/json

{
  "Attr1": 0.12,
  "Attr6": 1.50,
  "Attr35": -0.80
}
```

## Prediction Response

```
{
  "prediction": 1,
  "label": "Bankrupt",
  "probability": 0.41,
  "risk_level": "High",
  "risk_percentage": "41%",
  "trusted": false,
  "threshold": 0.29,
  "top_features": [
    {"feature": "Attr35", "value": -2.37, "importance": 0.1001}
  ],
  "history_id": "uuid-..."
}
```

**trusted:** true when probability  $\geq 0.60$  (the CONFIDENCE\_THRESHOLD config value). Below that, the prediction is made but flagged as lower-confidence.

**top\_features:** The top contributing features for this specific prediction, with their scaled values and model importance weights. Useful for understanding why the model flagged a company.

## 8. Frontend

Stack: React 18 + Vite 5 + Tailwind CSS 3 + Recharts 2. Live at <https://bankruptcy-sense-ai.vercel.app>

### Tab Structure

| Tab            | Components                                                  | Function                                                                         |
|----------------|-------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Predict</b> | PredictionForm, ResultCard, FeatureChart, AIRecommendations | Enter financial ratios, view probability gauge + feature breakdown + risk advice |
| <b>Batch</b>   | BatchUpload                                                 | Upload CSV, view summary statistics and per-row results                          |
| <b>History</b> | HistoryTable                                                | View, refresh, and clear past predictions                                        |

### Key Components

**RiskGauge:** Animated SVG semicircle gauge. The probability counter animates from 0 to the result value on render, giving clear visual feedback on severity.

**DashboardStats:** Radial bar charts displaying ROC-AUC, Recall, CV Std, and selected feature count — the four metrics that matter for this problem.

**AIRecommendations:** Generates risk-level-specific action plans from a local lookup table. This component works independently of the backend and does not make API calls.

### Environment Configuration

```
VITE_API_URL=https://bankruptcysense-ai.onrender.com
```

## 9. Deployment

### Backend — Render

| Setting        | Value                                                        |
|----------------|--------------------------------------------------------------|
| Root Directory | bankruptcy-predictor/backend                                 |
| Build Command  | pip install -r requirements.txt                              |
| Start Command  | gunicorn app:app --bind 0.0.0.0:\$PORT --workers 2 --timeout |



|              |                                        |
|--------------|----------------------------------------|
|              | 120                                    |
| Health Check | /health                                |
| CORS_ORIGINS | https://bankruptcy-sense-ai.vercel.app |

The model .pkl files (~13 MB total) are committed to the repository and cloned to Render at deploy time. No training runs on Render — only inference. Render auto-deploys on every push to main.

## Frontend — Vercel

| Setting        | Value                                     |
|----------------|-------------------------------------------|
| Root Directory | bankruptcy-predictor/frontend             |
| Build Command  | npm run build                             |
| Output Dir     | dist                                      |
| SPA Routing    | vercel.json rewrite: "/"* → "/index.html" |

## Updating the Deployed Model

```
# After retraining locally:
git add ml/model/*.pkl
git commit -m "Retrain: <describe what changed>"
git push origin main
# Render auto-deploys within ~2 minutes
```

# 10. How to Retrain

## Local Retraining

```
cd bankruptcy-predictor
# Ensure 5year.arff is in data/raw/
python ml/train.py
# Evaluate results
python ml/evaluate.py
```

## Google Colab (School Submission)

1. Open BankruptcySense\_AI\_Colab.ipynb in Colab
2. Run all cells (Runtime → Run all)
3. Upload 5year.arff when prompted
4. Download model\_artifacts.zip at the end
5. Extract and copy the five .pkl files to ml/model/

## Configuration Reference

All constants live in config.py. Change values there — never hardcode them elsewhere.

| Constant               | Value  | Purpose                                              |
|------------------------|--------|------------------------------------------------------|
| TOP_N_FEATURES         | 30     | Number of features to keep after selection           |
| TEST_SIZE              | 0.2    | 80/20 train/test split                               |
| MISSING_DROP_THRESHOLD | 0.40   | Drop columns with more than 40% missing              |
| OUTLIER_LOWER_PCTILE   | 1      | Clip below 1st percentile                            |
| OUTLIER_UPPER_PCTILE   | 99     | Clip above 99th percentile                           |
| N_ITER_SEARCH          | 30     | RandomizedSearchCV iterations                        |
| CV_FOLDS               | 5      | StratifiedKFold fold count                           |
| SCORING_METRIC         | recall | Metric to optimise during tuning                     |
| MIN_RECALL_FLOOR       | 0.75   | Minimum acceptable recall for threshold selection    |
| MIN_PRECISION_FLOOR    | 0.20   | Minimum acceptable precision for threshold selection |
| CONFIDENCE_THRESHOLD   | 0.60   | Probability above which result is marked "trusted"   |

## Appendix — Complete Training Log

```

=====
BankruptcySense AI - Training Pipeline
=====
[1/9] Loading data: Shape (5910, 65) | Bankrupt rate: 6.94%
[2/9] Dropping 1 column: ['Attr37']
[3/9] Train: (4728, 63) | Test: (1182, 63) | Bankrupt rate: 6.94%
[4/9] Median imputation (fit on train)
[5/9] Outlier clipping (1st-99th percentile, train-derived)
[6/9] StandardScaler (fit on train)
[7/9] SMOTE: {0: 4400, 1: 4400} (balanced)
[8/9] Top 30 features selected
[9/9] RandomizedSearchCV (30 iter, cv=5, scoring=recall)
      Best params: n_estimators=200, max_depth=None, class_weight=balanced
      Best CV recall: 0.9789
— CV Stability: [0.9727 0.9795 0.9852 0.9818 0.9750] Std=0.0045 ✓
— Threshold: OOF precision=0.30, recall=0.76, F1=0.43 → threshold=0.29
— Test-Set Results
  Threshold : 0.29
  ROC-AUC   : 0.9121 ✓ (target > 0.90)
  Recall    : 0.7927 ✓ (target > 0.75)
  F1 Score   : 0.4610 ✗ (target > 0.72 — dataset ceiling)
  Confusion Matrix:
  [[965, 135], [17, 65]]
— Artifacts saved: rf_model.pkl, scaler.pkl, features.pkl,
                  clip_bounds.pkl, threshold.pkl

```